
Design Document for Cow-puter Vision-Animal Detection on Dairy Farms

Version 0

Team 21

COMPSCI 4ZP6

Dr.Mehdi Moradi

January 23, 2026

Group Members	Student Number	Contact
Xianpi Duan	400381987	duanx14@mcmaster.ca
Xiaozhao Duan	400381647	duanx15@mcmaster.ca
Zicheng Li	400359321	li1276@mcmaster.ca
Bozhi Zhang	400327039	zhanb50@mcmaster.ca
Zhanxu Ye	400390561	yez69@mcmaster.ca

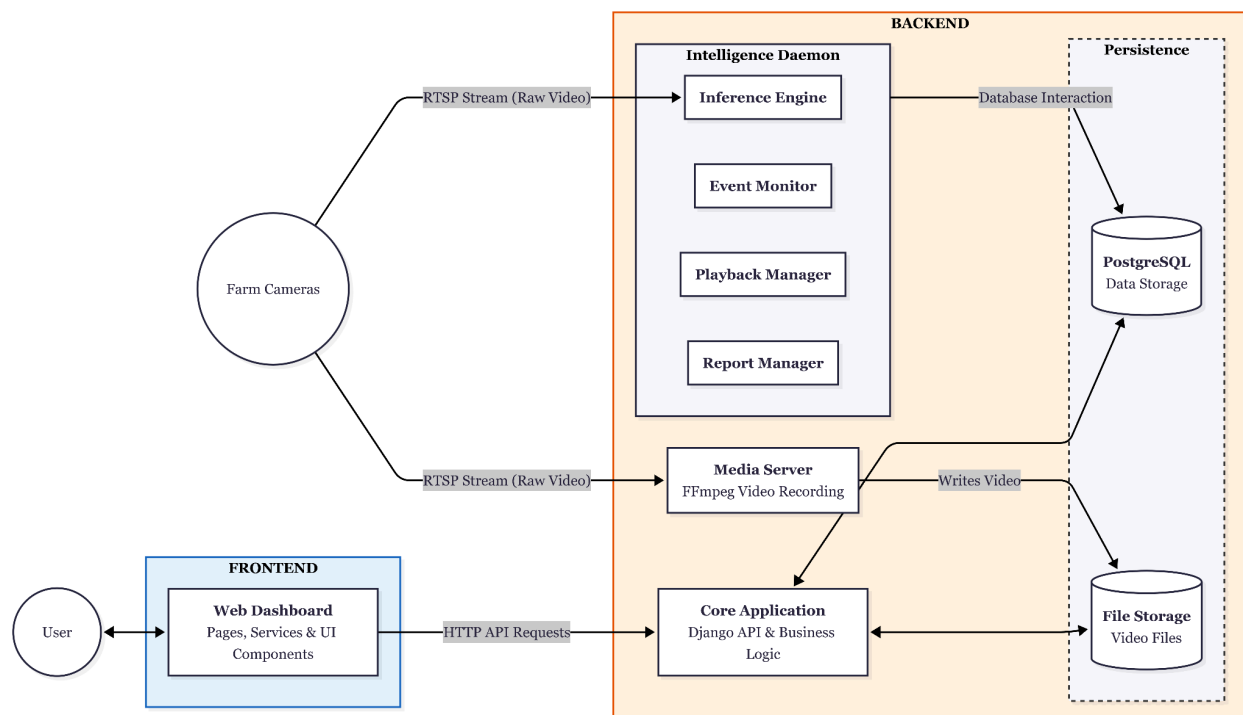
Design, Validation and Verification document for Cow-puter Vision-Animal Detection on Dairy Farms.....	0
Project Overview.....	1
Components.....	1
Relationship Between Project Components and Requirements.....	1
Frontend.....	2
Pages.....	2
Services and Schemas.....	4
UI.....	4
Backend.....	5
Architecture Overview.....	5
Components.....	5
The Core Application.....	5
The Intelligence Daemon.....	5
The Media Server.....	7
The Data Persistence Layer.....	7
Machine Learning Model.....	7
Cow Detection Module.....	8
Behavior Label Module.....	8
Cow Tracking Module.....	8
Module Relationships.....	9
Appendix.....	10
A. Backend.....	10
API Endpoint.....	10
Database ER Diagram.....	13
B. UI Design.....	14
C. Functional Requirements (Summarized from SRS).....	17

Project Overview

The purpose of this project is to design and implement a complete system that integrates machine learning, backend processing, and a frontend interface to detect, track, and analyze individual cows in video footage for automated behavior analysis. The purpose of this document is to present the overall system architecture, including the machine learning models, backend components, frontend interface, and the verification and validation plan used to ensure the system meets its functional and performance requirements.

Components

The system is composed of three main components: frontend, backend, and a machine learning model. The diagram below shows how these components interact.



Relationship Between Project Components and Requirements

The design document covers all P0 and P1 features, as well as most P2 features defined in the SRS.

System Component	Requirements Covered(See Appendix C)
Frontend /auth	P0-FE-01

Frontend /dashboard/overview	P0-FE-02
Frontend /dashboard/live-camera	P0-FE-03, P0-FE-04, P1-FE-01, P1-FE-04,
Frontend /dashboard/playback	P1-FE-02, P2-FE-02, P2-FE-03
Frontend /dashboard/alert	P1-FE-05, P1-FE-06
Frontend /dashboard/reports	P2-FE-01
Frontend /dashboard/settings	P0-FE-01
Backend /api/*	P1-BE-01, P1-BE-02, P1-BE-03, P1-BE-04, P2-BE-01, P2-BE-02, P2-BE-03
Backend Intelligence Daemon	P0-BE-01, P0-BE-02, P0-BE-03, P0-BE-05
Backend Media Server	P2-BE-03, P0-BE-04
ML Model Cow Detection Module	P0-ML-01
ML Model Behavior Label Module	P0-ML-02
ML Model Cow Tracking Module	P0-ML-03

Frontend

The frontend is the user-facing layer of the system, responsible for rendering the user interface and handling user interactions. To support scalability, extensibility, and maintainability, the frontend is designed using a modular architecture.

The frontend is composed of three primary parts:

- **Pages:** route-level views that users can access via URLs.
- **Components:** reusable, self-contained UI elements shared across pages.
- **Services:** services encapsulate communication with the backend and act as the interface between the frontend and backend systems.

Pages

Pages are implemented within the corresponding page.tsx files under the frontend\app\ folder, with folder names as sub routes' path.

Route Path	Detail
/auth/register	Normal Behaviour

	● Renders a registration form if the software is not previously registered. ● Performs client-side validation and submits credentials to the authentication service. ● On success, it redirects to the login page and displays a confirmation message. Error Handling ● Displays field-level validation errors for invalid or incomplete input.
/auth/login	Normal Behaviours: ● Renders a login form with required input fields, with the email field pre-filled and read-only. ● Performs client-side validation on user input. ● Submits credentials to the authentication service. ● On success, redirects the user to the dashboard page. ● Displays a link to initiate password reset after email verification. Error Handling: ● Handling by displaying an incorrect password error.
/dashboard; /dashboard/overview	Normal Behaviour ● Renders dashboard navigation tabs and handles tab-based navigation. ● Displays summary statistics (Detected Cows, Walking/Standing, Feeding, Resting). ● Renders real-time and historical behaviour charts. ● Displays a summary of recent alerts. Error Handling ● Displays a fallback or error state if data fetching fails.
/dashboard/live-camera	Normal Behaviour ● Renders a live camera video stream. ● Overlays detected cow bounding boxes and labels. ● Provides behaviour and confidence threshold filters. Error Handling ● Displays a fallback or error state if video or metadata fetching fails.
/dashboard/playback	Normal Behaviour ● Renders recorded camera video playback. ● Overlays detected cow bounding boxes and labels. ● Displays a timeline widget and behaviour filters. Error Handling ● Displays a fallback or error state if video or metadata fetching fails.

/dashboard/alerts; /dashboard/alert/{id}	Normal Behaviour • Displays all alerts in a table (Name, Description, Last Modified Time, Status). • Displays alert details in a modal overlay when selected.
/dashboard/reports; /dashboard/reports/{id}	Normal Behaviour • Displays all reports in a table (Name, Description, Created Time). • Displays report details in a modal overlay when selected.
/dashboard/settings	Normal Behaviour • Renders a settings view for managing system settings. • Persists changes via the settings service. Error Handling • Reverts to the last saved state when an update cannot be applied.

Services and Schemas

Services are implemented within the corresponding files under the frontend\schemas and frontend\services folder. Schemas are defined types that are shared across pages, components, and services to ensure consistency in data structures and interfaces.

Service	Related API Endpoints	Implementation
auth.ts	GET/POST /api/setup POST /api/auth	RESTful API call and return data.
live-camera.ts	POST /api/tracks GET /hls/{filename}	RESTful API call and return data. Then expose interfaces for pages and components to subscribe.
alerts.ts	GET/POST/DELETE /api/alerts	RESTful API call and return data.
reports.ts	GET /api/reports	RESTful API call and return data.
settings.ts	GET/POST/DELETE /api/settings	RESTful API call and return data.

UI

The user interface adopts a modern Clean Enterprise SaaS aesthetic tailored for Agri-Tech, featuring a Forest Green and Neutral Grey colour palette that reinforces the brand identity while ensuring visual comfort. Typography is standardized on Inter, utilizing weight hierarchies (Bold headers, Regular body) to maximize legibility across data-heavy dashboards. Design Details could be found under Appendix B.

Backend

Architecture Overview

The backend serves as the centralized processing hub for the system. It bridges the gap between raw, unstructured video data and behavior insights. Its primary objective is to ingest real-time RTSP streams from farm cameras, apply computer vision algorithms (YOLO) to extract behavioral metrics, and serve this data securely to the frontend dashboard.

Components

The backend is divided into four distinct core services which efficiently balance the high-throughput requirements of video processing with the structured data management of a web application.

The Core Application

- **Type:** Web Server (WSGI)
- **Tech Stack:** Django + Django REST Framework (DRF)
- **Responsibility:** Acts as the central source of truth. It manages user identity, business logics, and serves as the primary gateway for the frontend to retrieve structured data. It does not process video directly to ensure interface responsiveness.
- **API Endpoints:** Please refer the Appendix A for the details of endpoints
 - /api/setup
 - /api/auth
 - /api/tracks
 - /api/alerts
 - /api/settings
 - /api/playback
 - /api/reports
 - /hls/*

The Intelligence Daemon

- **Type:** Python Background Process
- **Tech Stack:** OpenCV + PyTorch/YOLO, Watchdog
- **Responsibility:** A dedicated high-performance service focused strictly on real-time processing. It ingests raw video, performs inference, and writes results to the database.

InferenceEngine
● Initializes the model, manages the main processing loop, and coordinates data flow between the VideoStream, Classifier, and Database. It ensures the system runs at a

steady frame rate.

- VideoStream Class: Handles RTSP stream reading using OpenCV's VideoCapture with FFmpeg backend.
- BehaviorClassifier Class: Loads YOLO models and performs inference on frames.
- DatabaseHandler Class: Manages high-frequency write operations to the tracking_data table.
- Potential issues include the latency of inference, not capturing latest frames, the misalignment of timestamps with the m3u8 timestamps generated by FFmpeg, and incorrect behaviors of the model.

EventMonitor

- Tracks state duration and triggers alerts based on rules defined in the database.
 - StateTracker Class: Maintains an in-memory state machine for each Track ID.
 - AlertRuleEngine Class: Represents and evaluates business rules loaded from the database.
 - NotificationService Class: Dispatches alerts when rules are violated.
- If service crashes, the in-memory state will be lost. In addition, tracking algorithms may assign a new ID to the same cow after occlusion.

PlaybackManager

- Manages the long-term storage of video segments for playback. It ensures video files are indexed by precise timestamps to allow synchronized playback with tracking data.
 - IndexParser Class: Reads index.m3u8 generated by the Archival Feed. It specifically extracts the #EXT-X-PROGRAM-DATE-TIME tag to obtain millisecond-precision start times.
 - MetadataSyncer Class: Scans the recording directory or watches for file events. Inserts new segments into the recordings table with start_ts, end_ts, and filename.
 - RetentionService Class: Deletes video files and corresponding database records older than X days or when disk usage exceeds a threshold.
- Race conditions might happen if we read a file while FFmpeg is still writing it.

ReportManager

- Generates periodic reports based on aggregated tracking data. It runs as a scheduled task (e.g., daily at midnight) based on settings to summarize key metrics.
 - ReportGenerator Class: Aggregates data from tracking_data table to compute metrics like total feeding time, average walking duration, etc.
 - ReportStorage Class: Saves generated reports into the reports table with metadata.

The Media Server

- **Type:** Background Shell Script (Managed by Systemd)
- **Tech Stack:** FFmpeg (Dual-Output HLS)
- **Responsibility:** Handles the robust transport and archival of video data. It is completely decoupled from Python to ensure recording continues even if the AI or Web App crashes.

Live Feed Generator

- Produces low-latency streams for real-time viewing. Through the live feed generator, the live feed now has a synced timestamp with InferenceEngine, It will help the frontend draw the bboxes at current time.
 - Format: HLS (.m3u8 + .ts)
 - Latency Tuning: -hls_time 1 (1-second segments), -hls_list_size 5, -tune zerolatency, -preset ultrafast.
 - Timestamping: -use_wallclock_as_timestamps 1 to ensure the HLS tags match real-world time.
- Output: hls/live.m3u8, hls/*.ts

Archival Feed Generator

- Produces an HLS playlist for long-term storage and playback.
 - Format: HLS (.m3u8 + .ts).
 - Duration: -hls_time 600 (10-minute segments) or user-defined duration.
 - Persistence: -hls_list_size 0. Keep all segments in the index.m3u8 list effectively creating a full recording history.
 - Naming: -hls_segment_filename "archive_%Y%m%d_%H%M%S.ts" (using strftime) for human-readable filenames.
 - Timestamping: -hls_flags program_date_time. This automatically injects the #EXT-X-PROGRAM-DATE-TIME tag into the playlist for every .ts segment, ensuring ms-level precision for the Playback Manager.
- Output: rec/index.m3u8 + .ts files.

The Data Persistence Layer

- **Tech Stack:** PostgreSQL
- **Responsibility:** Shared storage accessible by both the Core App (Django) and the Intelligence Daemon.
- **Table:** Please refer to the ER diagram in the appendix.

Machine Learning Model

The model first classifies the current behavior of each cow into one of eight predefined categories, including walking, standing, feeding, drinking, and lying. Multi-object tracking

is then applied to maintain consistent identities for individual cows across frames, allowing behavior predictions to be aggregated over time.

The system does not expose external APIs. Data exchange between components is handled internally through structured outputs passed between modules.

This project is implemented as a modular computer vision pipeline for dairy cow monitoring, consisting of three core components: detection, behavior labeling, and tracking. Each component has a clearly defined responsibility and interacts with others through well-defined data structures.

Cow Detection Module

- Responsibility: Detect cows in each image frame and determine their spatial locations.
- Method:
 - Input: RGB image frame
 - Process: A YOLOv12 object detector trained on a single class (“cow”) is used to perform inference. The model is executed via the Ultralytics YOLO command-line interface. Prior to training, all labels are preprocessed to ensure a single cow class.
 - Output: A list of detected cow bounding boxes with confidence scores.
- This module focuses solely on identifying the presence and location of cows and does not perform behavior analysis or identity assignment.

Behavior Label Module

- Responsibility: Assign a semantic behavior label to each cow based on time-aligned annotations.
- Method:
 - Input: Timestamp extracted from the image filename and the cow’s identity.
 - Process: The module queries cow-specific CSV files containing behavior annotations and maps numeric behavior codes to semantic labels (e.g., standing, lying, feeding, drinking).
 - Output: A behavior label corresponding to the given time.
- Behavior labeling is synchronized with visual data through timestamps and is independent of the detection process.

Cow Tracking Module

- Responsibility: Maintain consistent cow identities across consecutive frames.
- Method:
 - Input: Detection results (bounding boxes and confidence scores)

- Process: A tracking algorithm (ByteTrack integrated with YOLO) associates detections over time using spatial and motion consistency.
- Output: Tracked cows with persistent IDs and updated bounding boxes.
- This module enables multi-cow tracking in scenes where multiple animals appear simultaneously.

Module Relationships

- The system operates as a sequential pipeline:
 - **Detection** → **Behavior Labeling** → **Tracking**
- Detection provides spatial localization, behavior labeling adds semantic context, and tracking assigns persistent identities. Abnormal behavior inference is not yet implemented and is considered future work.

Potential Undesired Behaviours includes **False Negatives (FN)**: Cows may not be detected under occlusion, low lighting, lying postures, or when cows are very close together. **False Positives (FP)**: Background objects such as fences, shadows, or reflections may be incorrectly detected as cows. **Overlapping Cows**: Overlapping or clustered cows can lead to inaccurate or merged bounding boxes. **ID Switch**: Track IDs may be swapped when cows cross paths or experience temporary occlusion. **Track Loss**: A cow may lose its track during short occlusions or when leaving and re-entering the camera view, resulting in a new ID.

Appendix

A. Backend

API Endpoint

GET/POST /api/setup
• Check initial status and perform first-time setup. GET returns whether the app is initialized. POST performs setup; if database is empty, write the config to AppConfig, encrypt password, return initial JWT Token.; if not empty, return 403 Forbidden (app already initialized). ◦ Input: POST JSON payload with setup details {username, password, email, rtsp_url}. ◦ Output: GET return JSON with initialization status {"initialized": true/false}. POST return JSON with JWT token {"token": "<jwt_token>"} on success, or 403 error if already initialized. • It will be implemented as a Django REST Framework APIView. It will check the AppConfig model for existing entries, validate input data, hash passwords using Django's built-in utilities, and generate tokens using DRF's token authentication system. • Multiple setup attempts, invalid input data, database write failures might happen, and need to be carefully handled.
POST /api/auth
• Authenticate users and issue JWT tokens for session management. ◦ Input: JSON payload with credentials {username, password}. ◦ Output: JSON with JWT token {"token": "<jwt_token>"} on success, or 401 Unauthorized on failure. • This will be implemented using DRF's built-in authentication views, leveraging Django's authentication backend for user validation. • Potential issues include brute-force attacks, invalid credentials, and token generation failures.
POST /api/tracks
• Accepts a json logic, returns tracking data from the Intelligence Daemon which matches the logic. ◦ Input: JSON payload with logic rules. Fields include time range, behavior, and cow id. For example: { "and": [{ "===": [{ "var": "behavior" }, "feeding"] }, { ">=": [{ "var": "timestamp" }, 1769124135] }, { "<=": [{ "var": "timestamp" }, 1769124155] }] } ◦ Output: JSON array of tracking data objects matching the logic. For example:

```
[ { "cow_id": "cow_123", "behavior": "feeding", "timestamp": 1769124140,
  "bbox": [x1, y1, x2, y2] }, ... ]
```

- This will be implemented as a DRF APIView that parses the incoming logic, translates it into database queries, and retrieves matching records from the tracking data store.
- Potential issues include malformed logic, database query failures, and performance bottlenecks with complex queries.

GET/POST/DELETE /api/alerts

- Manages alert rules and retrieves triggered alerts. GET retrieves alerts, POST creates or updates alert rules, DELETE removes alert rules. Rules are defined using json logic.
 - **Input:** POST payload with alert rule details {rule_id, conditions, actions}, DELETE with {rule_id}.
 - **Output:** GET returns JSON array of alerts, POST returns success status, DELETE returns success status.
- This will be implemented using DRF's ModelViewSet to handle CRUD operations for alert rules and triggered alerts.
- Potential issues include invalid rule definitions.

GET/POST/DELETE /api/settings

- Manages application settings. GET retrieves current settings, POST updates settings, DELETE resets to defaults. Helpers will validate setting values.
 - **Input:** POST payload with settings {setting_name, value}, DELETE with no payload.
 - **Output:** GET returns JSON object of current settings, POST returns success status, DELETE returns success status.
- This will be implemented using DRF's ModelViewSet to handle CRUD operations for application settings.
- Potential issues include invalid setting values.

GET /api/playback

- Retrieves the .ts video segment corresponding to a specific timestamp range for playback.
 - **Input:** Query parameters with start and end timestamps (?start=1769124135&end=1769124155).
 - **Output:** Video file stream of the requested .ts segment.
- This will be implemented as a DRF APIView that calculates the appropriate .ts file based on the requested timestamps and streams the file back to the client.
- Potential issues include missing video segments, file access errors, and invalid timestamp ranges.

GET /api/reports

- Lists available reports, and retrieves specific report data.
 - **Input:** Query parameters for report identification (e.g., ?report_id=uuid). Or no parameters to list all reports.
 - **Output:** JSON array of available reports or JSON object with specific report data.
- This will be implemented as a DRF APIView that queries the reports data store and returns the requested information.

GET /hls/{filename}

- Serves HLS video segments generated by FFmpeg for real-time playback. {filename} corresponds to the requested .m3u8 playlist or .ts segment.
 - **Input:** HTTP GET request for HLS files.
 - **Output:** HLS file stream (.m3u8 or .ts).
- This will be implemented using Django's static file serving capabilities, mapping the /hls/ URL to the directory where FFmpeg outputs HLS files.
- Potential issues include missing files, file access errors.

Database ER Diagram

User		
int	id	PK
string	username	
string	email	
string	password_hash	
boolean	is_staff	

AppConfig		
int	id	PK
boolean	is_initialized	
string	rtsp_url	

Setting		
string	key	PK
string	value	

VideoSegment		
int	id	PK
string	filename	
bigint	start_ts	
bigint	end_ts	
string	file_path	

TrackingData		
bigint	id	PK
string	cow_id	
bigint	timestamp	
string	behavior	
json	bbox	

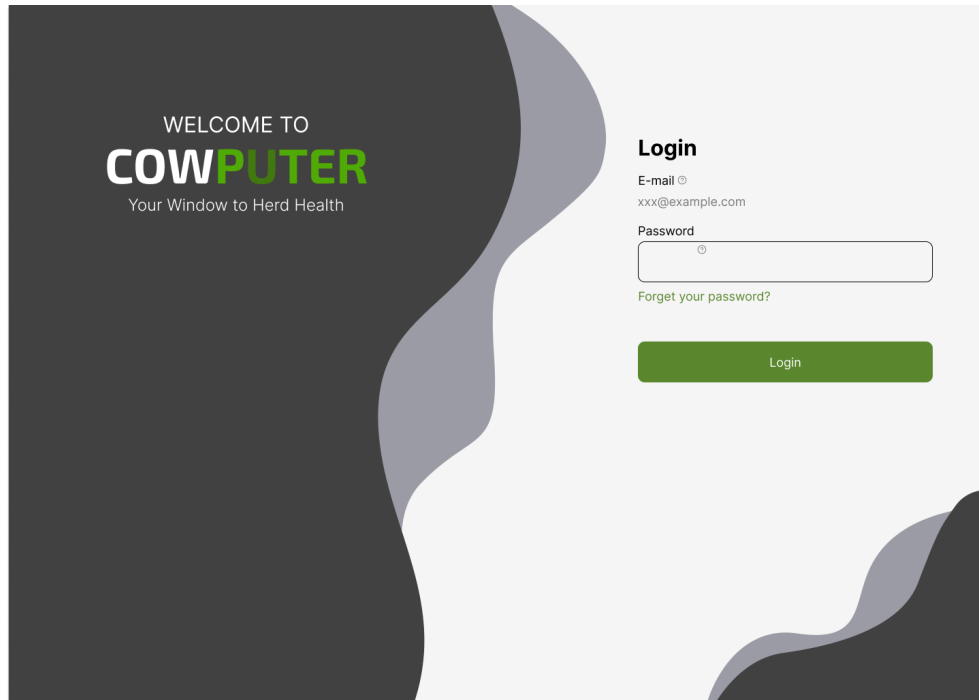
Report		
uuid	report_id	PK
string	report_type	
json	data	
datetime	generated_at	

AlertRule		
int	id	PK
string	name	
json	conditions	
json	actions	
boolean	is_active	

AlertEvent		
int	id	PK
int	rule_id	FK
datetime	triggered_at	
json	details	



B. UI Design



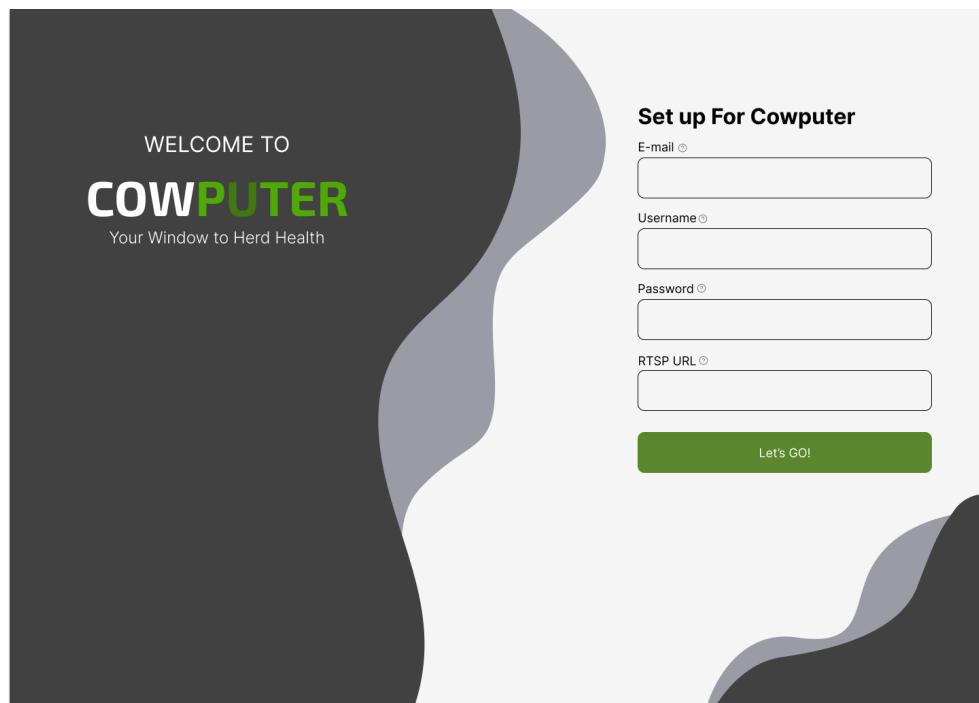
WELCOME TO
COWPUTER
Your Window to Herd Health

Login

E-mail ⓘ
xxx@example.com

Password ⓘ

[Forget your password?](#)



WELCOME TO
COWPUTER
Your Window to Herd Health

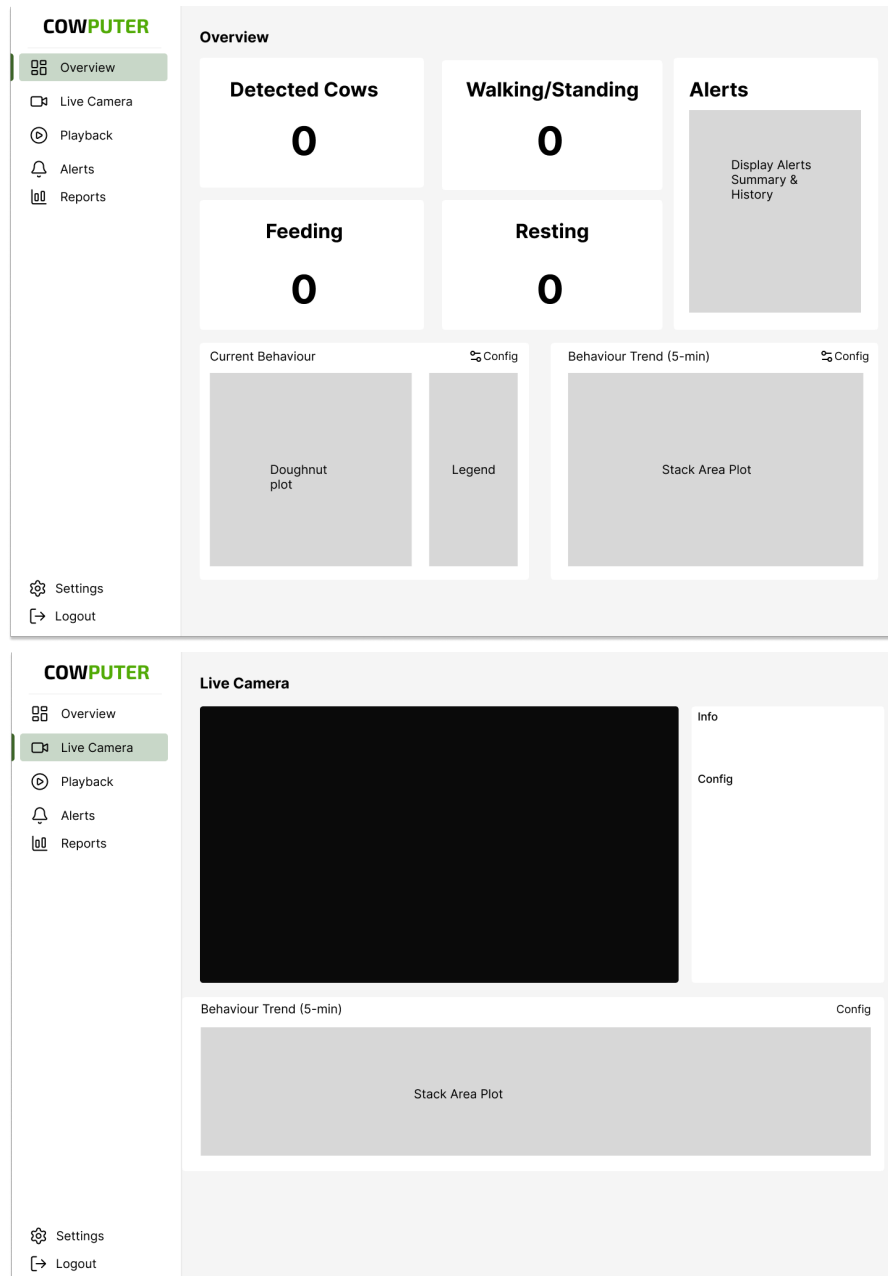
Set up For Cowputer

E-mail ⓘ

Username ⓘ

Password ⓘ

RTSP URL ⓘ



COWPUTER

Overview

Live Camera

Playback

Alerts

Reports

Settings

Logout

Playback

COWPUTER

Overview

Live Camera

Playback

Alerts

Reports

Settings

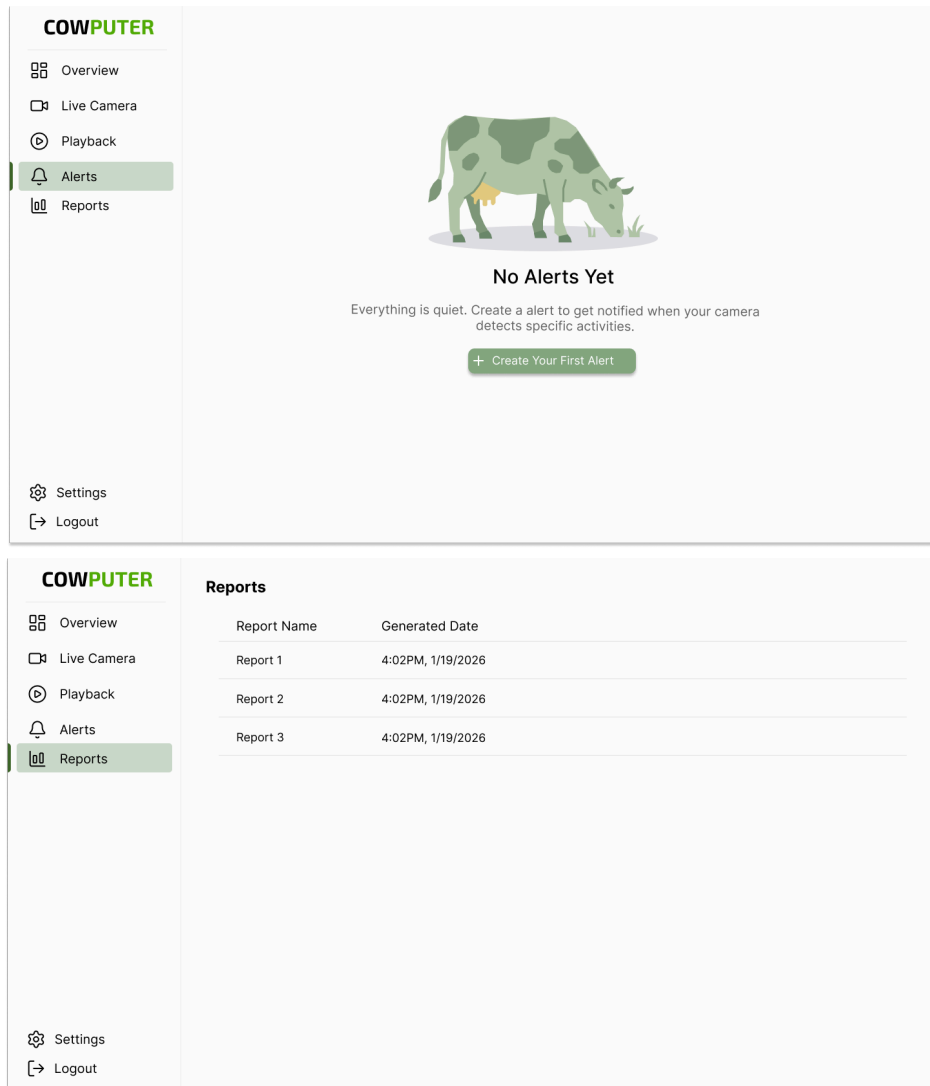
Logout

Alerts

+ New Alert

Bulk Actions

☐	Name	Description	Last Modified Time	Status
☐	Alerts 1	Some Description...	4:02PM, 1/19/2026	Enabled
☐	Alerts 2	Some Description...	4:02PM, 1/19/2026	Enabled
☒	Alerts 2	Some Description...	4:02PM, 1/19/2026	Disabled



C. Functional Requirements (Summarized from SRS)

P0-ML-01	P0	ML	Detect cows in a video frame
P0-ML-02	P0	ML	Classify cow behaviour (Standing, Lying Down, Feeding, Drinking)
P0-ML-03	P0	ML	Track detected cows across consecutive video frames
P0-BE-01	P0	BE	Receive video streams from cameras
P0-BE-02	P0	BE	Perform model inference on received video frames
P0-BE-03	P0	BE	Provide video streams with inference outputs to the frontend
P0-BE-04	P0	BE	Store received video frames in the local file system
P0-BE-05	P0	BE	Store video frame paths and corresponding inference outputs
P0-FE-01	P0	FE	Provide a web-based frontend requiring user authentication
P0-FE-02	P0	FE	Play video streams

P0-FE-03	P0	FE	Overlay bounding boxes and labels on video playback
P0-FE-04	P0	FE	Display a dashboard of cows' current behaviours and durations
P1-BE-01	P1	BE	Provide APIs to retrieve behaviour data with filtering
P1-BE-02	P1	BE	Provide CRUD APIs for alerts
P1-BE-03	P1	BE	Send proactive notifications for triggered alerts
P1-BE-04	P1	BE	Store alert triggering conditions and relevant data
P1-FE-01	P1	FE	Filter overlaid bounding boxes by behaviour labels
P1-FE-02	P1	FE	Display detailed behaviour data for each detected cow
P1-FE-03	P1	FE	Display aggregated behaviour visualizations for all cows
P1-FE-04	P1	FE	Display behaviour visualizations for a single cow
P1-FE-05	P1	FE	Configure alerts with custom conditions and actions
P1-FE-06	P1	FE	Display configured alerts sorted by priority and timestamp
P1-FE-07	P1	FE	Display triggered alerts with conditions and data
P2-BE-01	P2	BE	Provide playback of stored video with inference outputs
P2-BE-02	P2	BE	Aggregate data for report generation
P2-BE-03	P2	BE	Generate video files with stored inference outputs
P2-FE-01	P2	FE	Provide a reporting section with customizable date ranges
P2-FE-02	P2	FE	Provide a playback section
P2-FE-03	P2	FE	Export and download overlaid videos